



Exercise 3

Deadline: 8 December, 4pm

This exercise does not contribute to the module mark.

The Task

Write an extension to the linux firewall which makes it possible to specify which programs are allowed use which outgoing port.

More precisely, you should write a user space program and a kernel module.

Firewall rules

A firewall rule consists of a port number and a filename (the full path) of a program separated by a space, meaning that the corresponding program is allowed to make outgoing connections on this TCP-port. If there is no rule for a given port, any program should be allowed to make outgoing connections on this port. A connection is not allowed when rules for the port exist, but the program trying to establish the connection is not in the list of allowed programs. If a connection is not allowed, it should be immediately terminated.

The kernel module processes the packets and maintains the firewall rules, and displays the firewall rules via `printk` in `/var/log/kern.log`. The output should be:

Firewall rule: `<port>` `<program>`

For every rule that is configured, `<port>` is the port number in decimal representation and `<program>` is the full path to the executeable.

When the kernel module is unloaded, the firewall extensions should be deleted.

User space configuration

The user space program, which must be called `firewallSetup` and be placed in a directory `Setup`, has commands firstly for triggering the listing of the firewall rules in `/var/log/kern.log`, and secondly for setting the firewall rules. A new set of firewall rules overrides the old set (no appending). You should use the file `/proc/firewallExtension` for communication between the user program and the kernel.

If replacing the set of firewall rules fails for any reason, the old set of firewall rules should be retained.

To make marking easier, there should be two ways of calling the user space program. The first one is

`firewallSetup L`

This way of calling the user space program causes the firewall rules to be displayed in `/var/log/kern.log` as specified above.

The second way of calling the program is

`firewallSetup W <filename>`

where `<filename>` is the name of the file containing the firewall rules. This file contains one firewall rule per line. `firewallSetup` should check whether the filename in the firewall rule denotes an existing executable file. If there is any error in the syntax or any filename does not denote an executable file, this program should abort with the messages `ERROR: Ill-formed file` and `ERROR: Cannot execute file` respectively.

Submission

You should submit a zip-file on canvas containing a directory called `ex3`. This directory should contain all the sources. The directory `ex3` should in addition contain a Makefile which builds the kernel module which must be called `firewallExtension.ko`. This directory must also contain a subdirectory called `Setup`. This subdirectory must contain a Makefile which builds the `firewallSetup`-binary. We will run the command `make` in the directory `ex3` in order to obtain the kernel module, and we will run `make` in the subdirectory `Setup` to obtain the `FirewallSetup`-binary. Do not use any files or directory with the prefix `test`, since we will use such files for running tests.

Hints

- There is an archive containing the source for two kernel modules. The first one, `findExecutable` provides code which is useful for finding the full path of the executable. The code outputs the filename of the executable without the directory part, and the filename of the directory (without its directory part) which contains the file. As an example, if the full path was `/usr/bin/gcc`, the output would be `gcc` and `bin`.
- The second module is a module which modifies the firewall in such a way that it rejects all outgoing connections to port 80. The code defines a function `FirewallExtensionHook` which is added to the packet filter in `init_module`. This function is called for each outgoing packet. This function checks first whether the packet is an initial packet for a new connection. If the code reaches line 81, we know that this is the case, and that the full executable can be found via the method described in `findExecutable`.
- You can create outgoing connections via

`nc <hostname> <port>`

where `hostname` and `<port>` are the hostname and port on the destination. An example would be

```
nc localhost 22
```

- Visual Studio needs a different environment setup for kernel programming. The directory containing the kernel module already contains a suitable `.vscode`-subdirectory. These settings are not suitable for editing the user space program. Hence you need to edit the user space program in a different window and do not open it from the window where you have already opened the kernel module.

General Coding

Your kernel code may assume that only well-formed files are written by `firewallSetup`.

Only one process should be allowed to open the `/proc/firewallExtension` file at any give time. If a second process tries to open this file it should receive the error message `-EAGAIN`.

You need to ensure that you handle concurrency in the kernel correctly. In particular, any number of outgoing connections may be started at any time, hence several instances of the procedures handling the packets may be executed at the same time. It is very important that you maximize the degree of parallelism. In particular, your critical sections should be as short as possible.

We have provided a test script for development in the file `test.sh`